

云原生、AI Infra 与开源参与

从 Docker、Kubernetes、CNCF 生态，到学生能参与的真实开源协作

面向大学生的开源与云原生技术分享

今天按两条线展开

- 01 第一部分：CNCF、Docker、Kubernetes 与 AI Infra
- 02 Shiming 分享：Vibe Coding 如何改变开发体验
- 03 第二部分：开源机制、收益与参与实践
- 04 最后：学习路径、思考题与 Q&A

讲者背景：长期在 Kubernetes 和开源社区工作

徐俊杰 / pacoxu

- Kubernetes 指导委员会成员。
- CNCF TAG Workloads Foundation Chair 主要参与 kubeadm、SIG Node、Kubernetes 发行与社区治理。
- Linux Foundation APAC 开源布道师。
- GitHub: pacoxu

Kubernetes / kubeadm / SIG Node

d.run 全景图：AI Infra 需要把算力、模型和 Token 连起来

一个企业级 AI Infra 样本

d.run 连接异构算力、Kubernetes 调度、推理优化、模型服务、调用治理和业务场景。学生先抓三层：算力硬件、调度底座、上层模型服务。

看图时只抓三层：算力硬件、Kubernetes 调度底座、上层模型服务。



Part 01

云原生与 AI Infra

先理解现代软件为什么需要新的基础设施



软件交付的难题，不是从云开始的

- 本地能跑，服务器不能跑：环境、依赖、配置都可能不同。
- 用户流量变化快：应用需要快速扩容、缩容、发布和回滚。
- 系统越来越复杂：一个大应用被拆成很多服务后，人工运维成本急剧上升。

从物理机到虚拟机，再到容器

01

物理机

一台机器部署一个或少量应用

02

虚拟机

更强隔离，但镜像重、启动慢

03

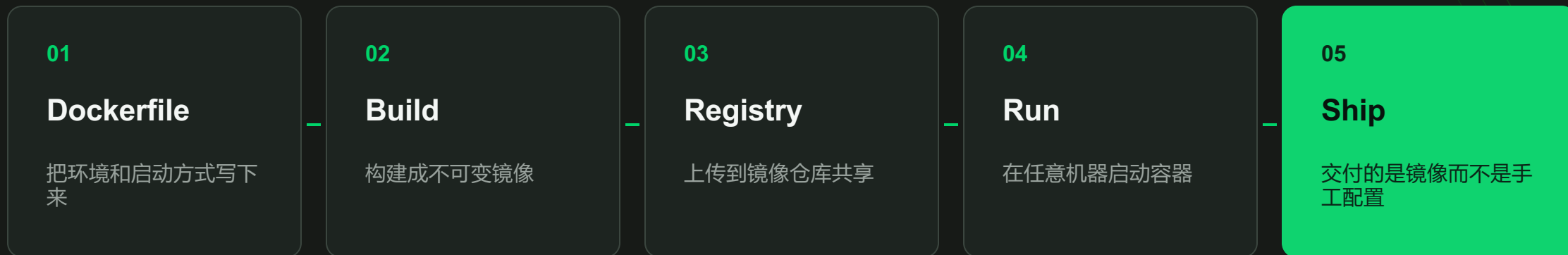
容器

应用和依赖一起打包，交付更轻

一次构建 到处运行

容器让应用交付从“配置一台机器”变成“交付一个镜像”

Docker 把容器变成一套开发者 workflow



Docker 的关键价值不是“多一个启动命令”，而是把开发、测试、发布都围绕同一个镜像展开。

先分清镜像、容器和仓库

Image / 镜像

应用、依赖和启动配置打包后的只读模板。它像一个可复制的软件制品。

Container / 容器

镜像运行起来后的进程。它有隔离的文件系统、网络和资源边界。

Registry / 仓库

存放和分发镜像的地方。团队通过仓库共享版本化的软件制品。

简单记法：镜像是“包”，容器是“运行中的包”，仓库是“存包和分发包的地方”。

Dockerfile 把环境写成可复现的说明书

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

FROM 选择基础环境

RUN 安装依赖和工具

COPY 把代码放进镜像

CMD 说明容器启动后做什么

Docker 常用命令按生命周期记

docker build

根据 Dockerfile 构建镜像

docker run

从镜像启动一个容器

docker ps / logs

查看正在运行的容器和日志

docker exec

进入容器执行调试命令

**docker pull /
push**

从仓库拉取或上传镜像

docker stop / rm

停止并清理容器

先理解生命周期：构建镜像、运行容器、查看状态、调试、推送和清理。

Docker 和虚拟机解决的问题不同

虚拟机

每个虚拟机都有完整操作系统，隔离强，但镜像更重、启动更慢。



容器

容器共享宿主机内核，隔离在进程和资源层，镜像更轻、启动更快。



容器不是“更小的虚拟机”。它把应用隔离和交付做轻，但集群级调度仍然需要 Kubernetes。



镜像仓库让软件制品可以被管理

- 代码进入 Git，容器镜像进入 Registry。
- 镜像有版本、标签、扫描、签名和回滚需求。
- 软件供应链安全从这里开始变得重要。



容器解决一致性，但不负责管理集群

- 一台机器上跑容器很简单，几百台机器就复杂了。
- 谁来决定容器放在哪台机器？机器坏了怎么办？流量怎么进来？
- 这就是容器编排系统要解决的问题。

命令式和声明式的区别，决定了平台怎么工作

同一个目标：让应用始终运行 3 个副本

命令式 Imperative

告诉系统一步步怎么做

- 1 登录机器 A，启动 1 个进程
- 2 登录机器 B，再启动 1 个进程
- 3 机器坏了，人工找新机器重启
- 4 扩容时继续手工选择机器

你负责记住过程和处理例外

声明式 Declarative

告诉系统最终想要什么

- 1 声明 replicas = 3
- 2 系统观察当前只有几个副本
- 3 自动创建、迁移或重启 Pod
- 4 持续把现实拉回期望状态

系统负责执行过程和持续校正

Kubernetes 偏声明式：用户提交期望状态，控制器持续调谐现实状态。

Kubernetes 的核心思想：声明期望状态

01

声明

我希望有 3 个副本

02

观察

控制器检查当前状态

03

调谐

创建、删除或迁移 Pod

04

持续

系统不断接近期望状态



几个对象就能描述一个应用的基本运行

- Pod: Kubernetes 调度和运行的最小单元。
- Deployment: 描述副本数、升级策略和回滚。
- Service: 给一组 Pod 提供稳定访问入口。
- ConfigMap / Secret: 把配置和敏感信息从镜像中分离出来。

Pod 生命周期：从创建到清理是一条状态链



Pod 的状态要分三层看

Pod Phase

Pending / Running
Succeeded / Failed
Unknown
回答：Pod 到哪一步。

Container State

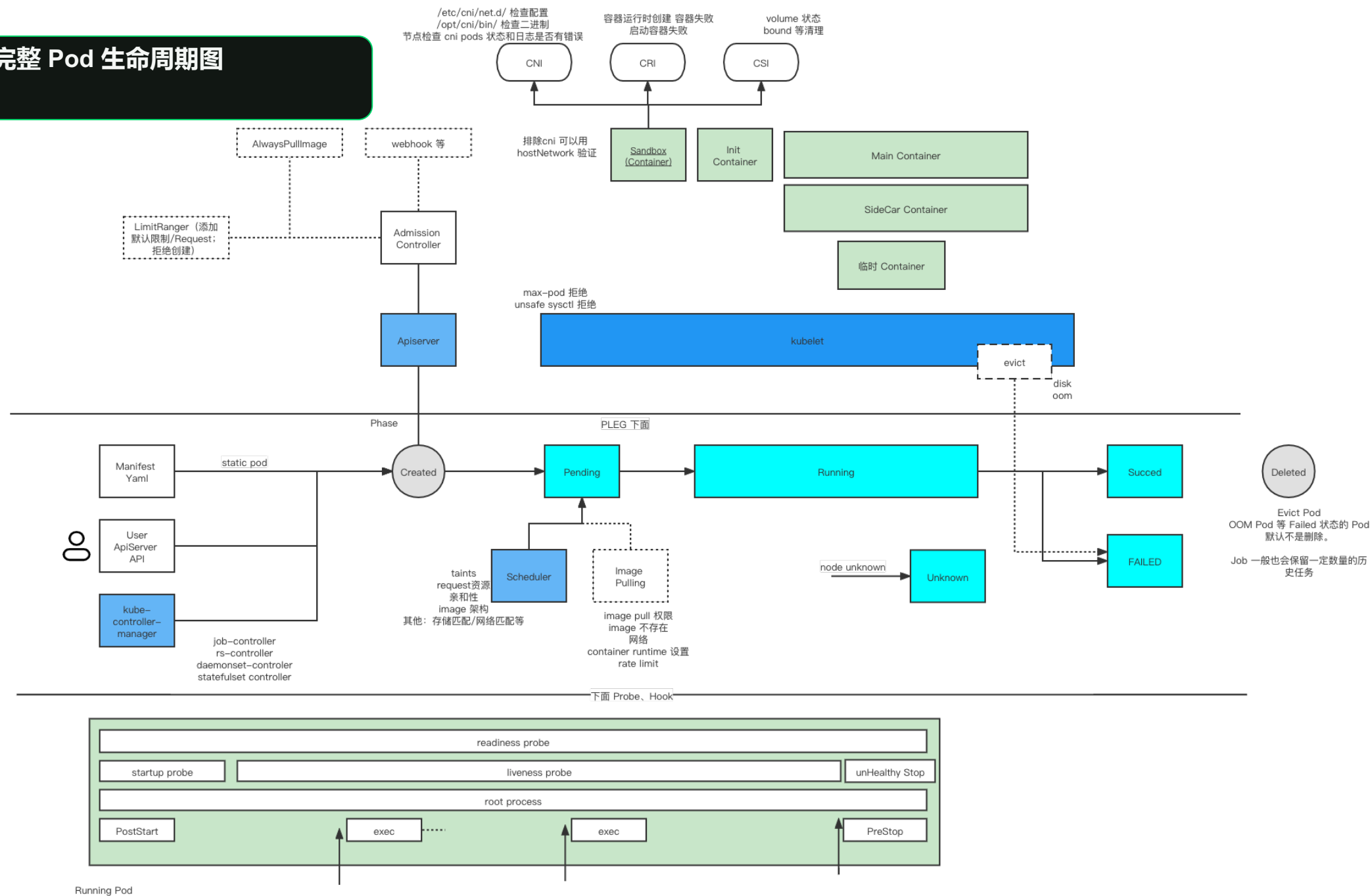
Waiting / Running
Terminated
结合事件和 restartCount
定位失败原因。

Probe 和 Hook

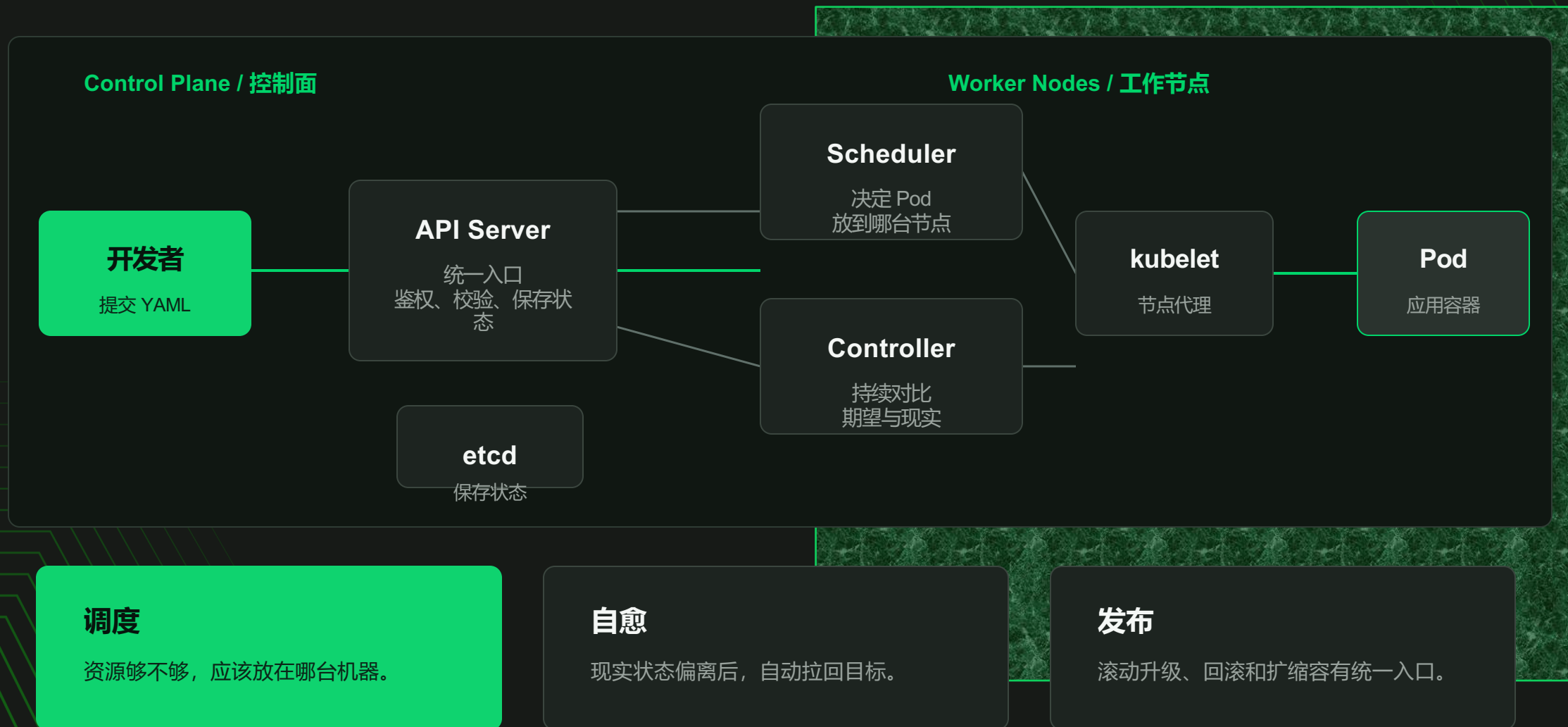
Probe：启动、存活、可服务
Hook：启动后、停止前
影响应用准备和退出。

关键提醒：Running 不等于 Ready。Pod 已经运行，不代表应用已经能接流量。

完整 Pod 生命周期图



Kubernetes 的基本架构：控制面让集群接近期望状态

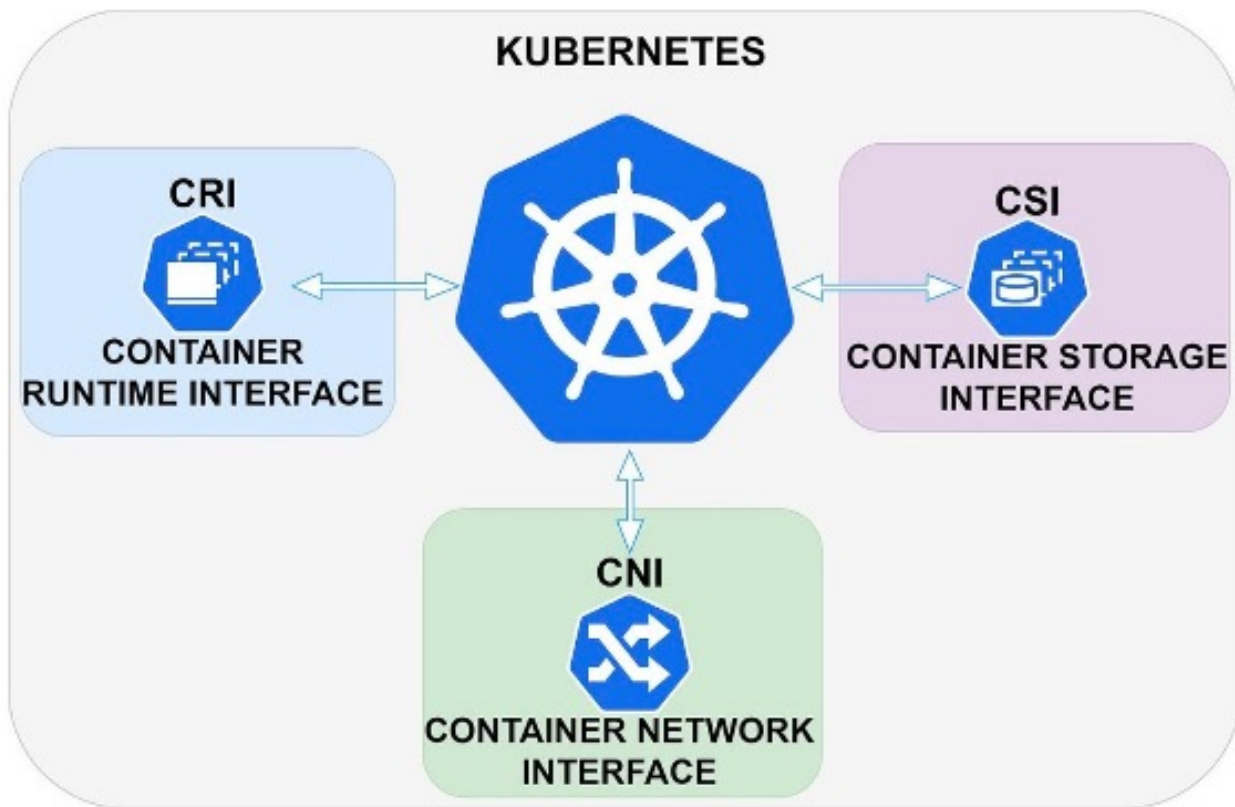


Kubernetes 帮平台处理大量重复问题

- 调度：把工作负载放到合适的机器。
- 自愈：失败后自动重建。
- 弹性：根据负载扩缩容。

- 发布：滚动升级、回滚、灰度。
- 扩展：通过 CRD、Operator、Webhook 扩展能力。

Kubernetes 成为标准，靠的是稳定核心和开放扩展



Kubernetes 可扩展性

1. Kubeadm 安装最小功能集
2. Admission Controller/Webhook
3. **CRD and Operator**
4. API Aggregation Layer
5. Device Plugin 设备管理
6. Kubectl krew plugins CLI
7. Scheduling Extension 调度
8. DRA 动态资源分配
9. KCP 控制平面
10. Gateway API 网关
11. ClusterProfile/Inventory API 多集群

为什么 Kubernetes 能成为事实标准

- 声明式 API 和控制循环，让平台可以自动接近期望状态。
- CRI、CNI、CSI、CRD、Operator 等扩展点让生态不用修改核心就能创新。
- CNCF 的中立治理让项目不只绑定某一家云厂商。
- 云上、机房、边缘都能使用同一套抽象，降低迁移和协作成本。



CNCF 给云原生项目提供中立协作场域

- CNCF 是 Linux Foundation 旗下基金会，承载 Kubernetes 等云原生项目。
- 基金会提供治理、商标、项目生命周期和社区协作框架。
- 项目不只属于某家公司，才能形成更大的产业生态。

CNCF 的 TOC 负责守住技术方向和中立流程

CNCF 不只是托管代码，也提供中立的技术治理框架

TOC / Technical Oversight Committee

看全局方向，维护项目生命周期，保证项目提案流程公平中立

TAG / 技术咨询小组

按领域聚集专家，沉淀共识，帮助 TOC 和项目做技术判断。

技术方向

哪些能力值得进入生态？

中立流程

项目不是某家公司说了算

Projects / 云原生项目

Kubernetes、Prometheus、Envoy 等项目按生命周期成长。

社区规模化

专家分散到不同领域协作

TAG 是按技术领域组织起来的专家社区

TAG 像“技术领域小组”：组织共识、评审和协作入口

技术领域

例如开发体验、基础设施、运行韧性、安全合规、工作负载基础。

共同产出

白皮书、技术评审、最佳实践、会议、工作组和社区活动。

连接项目

帮助项目找到领域专家，也帮助新贡献者找到可参与的入口。

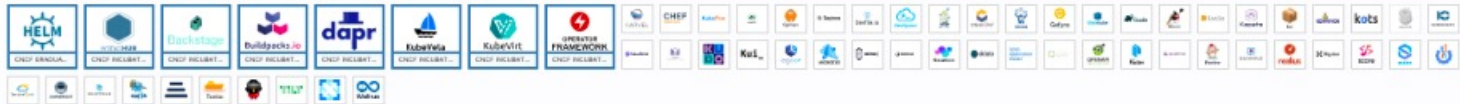
降低门槛

学生可以从参会、记录、文档、调研和翻译开始。

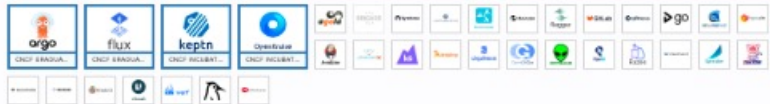
记法：TOC 看全局方向，TAG 看领域问题，Project 解决具体实现

App Definition and Development

Application Definition & Image Build 🔍



Continuous Integration & Delivery 🔍



Database 🔍



Streaming & Messaging 🔍



Orchestration & Management

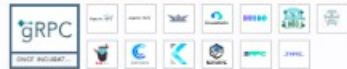
Scheduling & Orchestration 🔍



Service Proxy 🔍



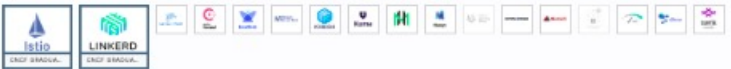
Remote Procedure Call 🔍



API Gateway 🔍



Service Mesh 🔍



Coordination & Service Discovery 🔍



Runtime

Cloud Native Storage 🔍



Cloud Native Network 🔍



Container Runtime 🔍



Provisioning

Security & Compliance 🔍



Key Management 🔍



Automation & Configuration 🔍



Container Registry 🔍



Observability and Analysis

Observability 🔍



Feature Flaggi... 🔍



Chaos Engineering 🔍



Continuous Optimization 🔍



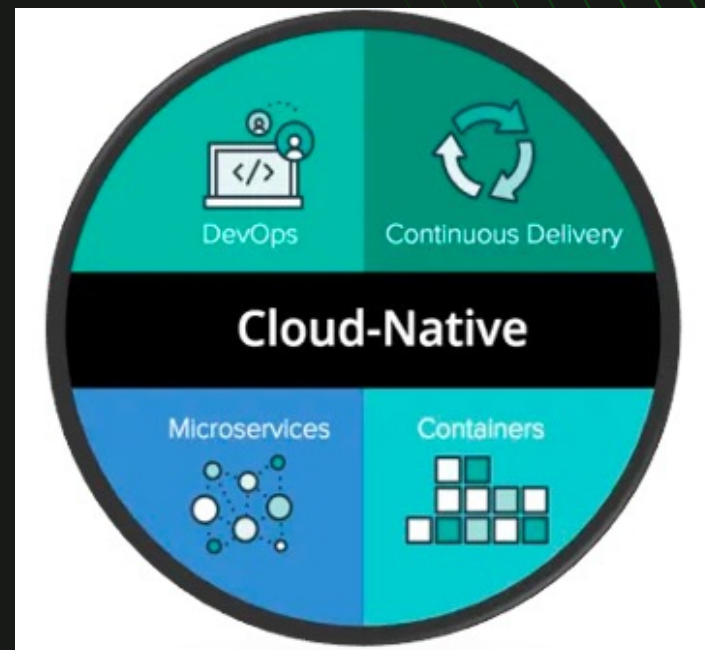


看 CNCF Landscape, 不用背项目名

- 先按问题分类：运行时、编排、网络、存储、可观测、安全、CI/CD。
- 再看项目成熟度：Sandbox、Incubating、Graduated。
- 最后结合使用场景判断：我现在到底要解决什么问题？

云原生常和三件事一起出现

- 容器化：应用和依赖可复制、可分发。
- 微服务：系统拆成更小的服务，便于独立演进。
- DevOps：开发、测试、运维、安全在同一交付链路中协作。



容器化

让应用可复制、可分发、可回滚。

微服务

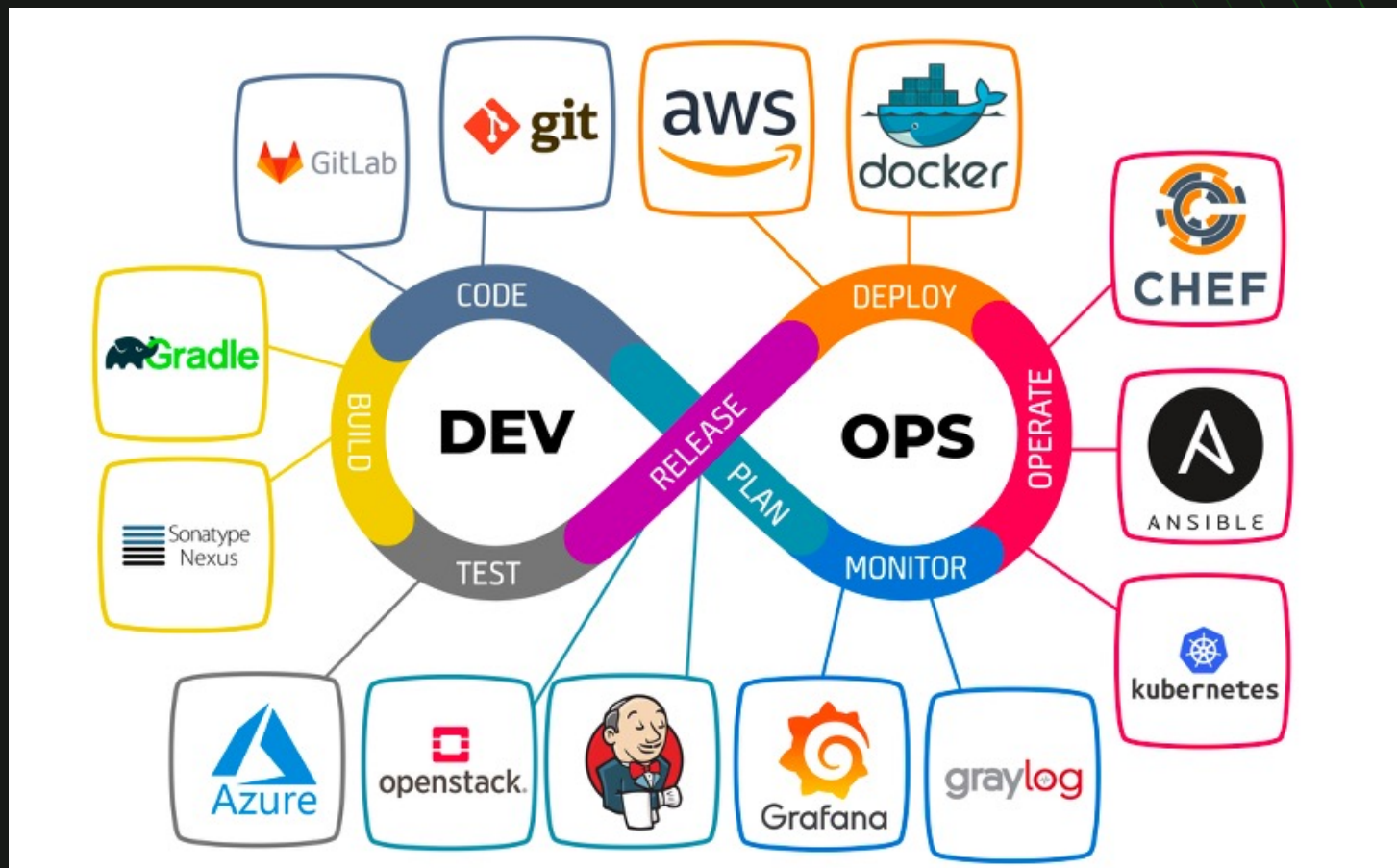
让系统按服务边界独立演进。

DevOps

让开发、测试、运维形成快速反馈。

DevOps 的重点不是工具，而是反馈速度

- 代码提交后自动构建、测试、扫描和发布。
- 问题越早发现，修复成本越低。
- 云原生平台让环境创建、部署和回滚都更自动化。





微服务让系统更灵活，也让治理更复杂

- 好处：独立发布、独立扩缩容、团队边界更清晰。
- 代价：调用链更长、配置更多、排障更难。
- 所以需要服务发现、网关、可观测、限流、熔断和发布治理。



渐进式发布降低变更风险

- 滚动发布：逐步替换旧版本。
- 灰度发布：先让少量用户使用新版本。
- 回滚：指标异常时尽快退回稳定版本。



可观测性回答：系统为什么出问题

- 日志 Logs：发生了什么。
- 指标 Metrics：系统状态是否异常。
- 链路 Tracing：一次请求经过了哪些服务。
- 云原生系统越复杂，可观测性越像基础设施。

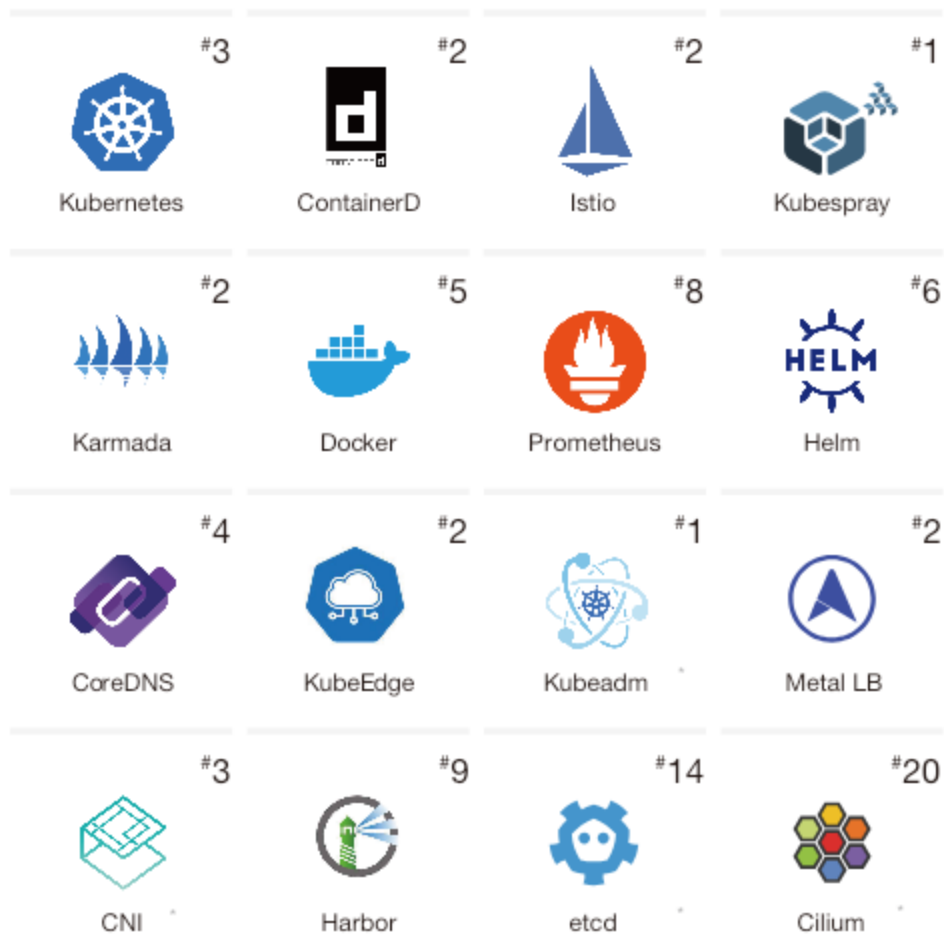


安全从镜像、配置、运行时一路贯穿

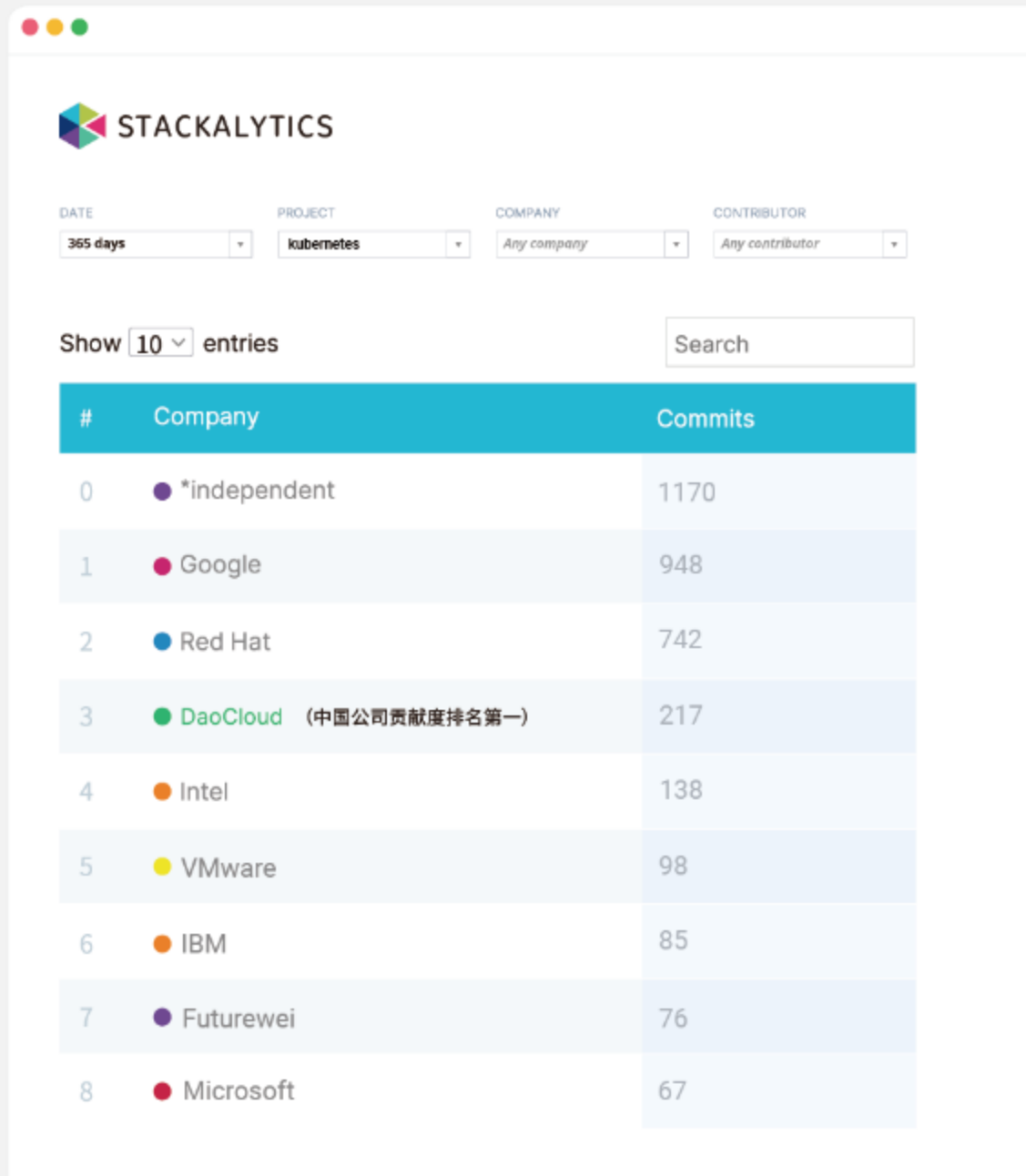
- 镜像来源是否可信？依赖是否有漏洞？
- 密钥是否被写进代码或镜像？
- 服务之间是否有最小权限和网络边界？
- 开源组件越多，供应链治理越重要。

DaoCloud

持续贡献全球顶级开源项目
掌握核心代码话语权



本页数据为「DaoCloud 道客」参与的开源项目在过去一年内的贡献度（GitHub Commits 数量）公司排名。大部份数据来源于 stackalytics.io，少数为人工统计。



AI 更多运行在 Kubernetes 上，平台问题也随之改变

- 过去平台主要运行 Web 服务、微服务和数据服务。
- 现在还要运行训练、推理、RAG、Agent 和模型网关。
- 这些工作负载更依赖 GPU、拓扑、缓存、队列、成本和隔离。

AI Infra 正在进入云原生阶段

CNCF 认为：云原生正从应用运行底座升级为 AI 工程化、规模化、治理化底座

Kubernetes 正在从容器编排平台，演进为 AI Infra 的基础操作系统。



数据与验证

CNCF 调研：约 66% 组织
已用 Kubernetes 承载 Gen AI
AI Conformance：DaoCloud 与
Google、NVIDIA、Azure 等第一批通过

产业案例



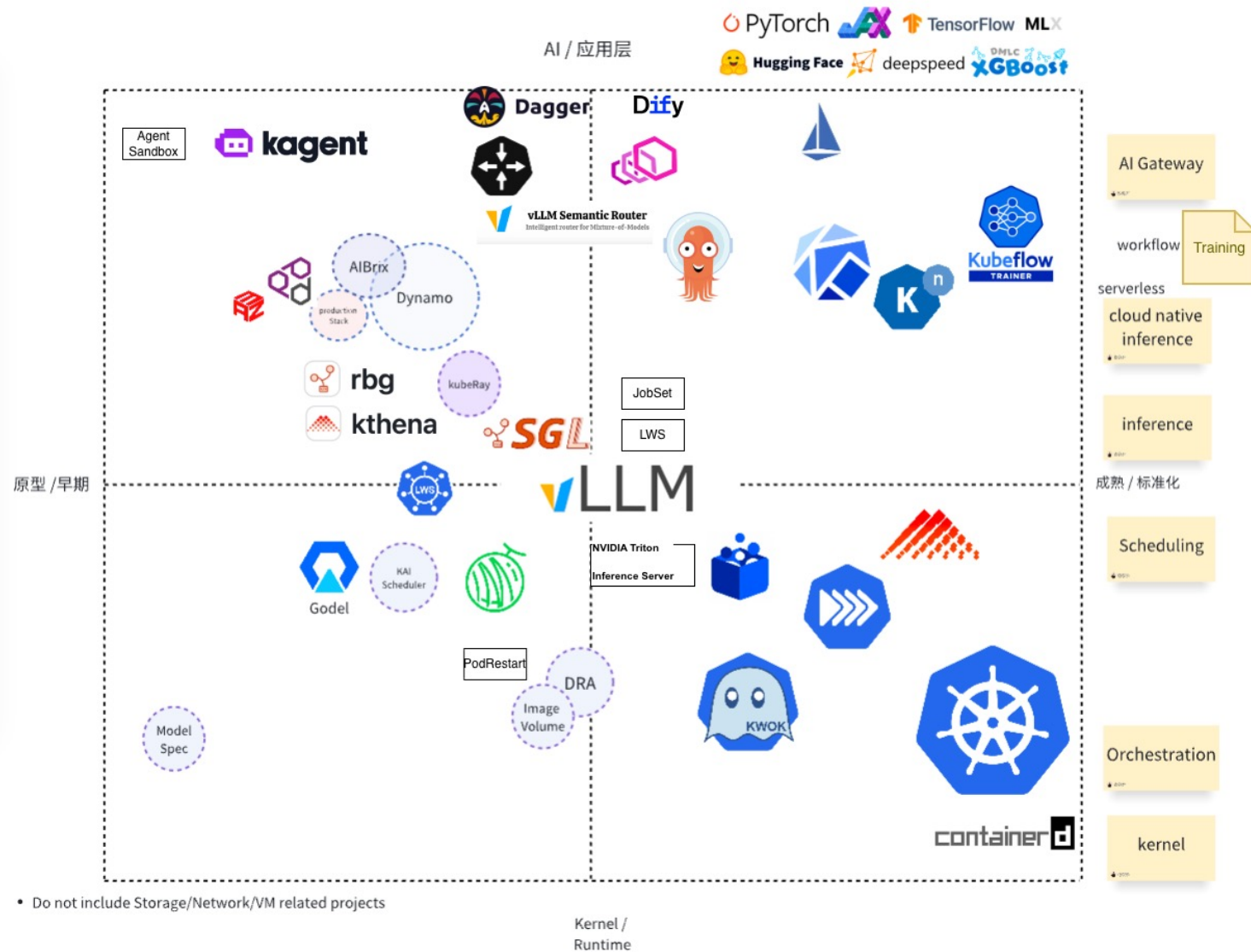
- 1 Google GKE / Anthropic：单集群 **13万节点**；2024 年支持 **25万卡**；Hypercluster 超 **100万卡**
- 2 AWS EKS / Anthropic / Nova：单集群 **10万节点**；**80万** 英伟达 GPU / **160万** Trainium
- 3 字节跳动：多集群 **50万+节点**；批调度单集群 **2万节点**；AI 训练公开到 **1.2万卡**
- 4 蚂蚁 / 阿里：Kubernetes **超万节点** 实践；训练 / 推理统一调度；GPU 池化
- 5 DeepSeek / High-Flyer：HAI Platform 基于 K8s；Fire-Flyer 2 达 **1万 A100**；V3 为 **2048 张 H800**
- 6 美团 LongCat-2.0：1.6T 参数 MoE 模型；使用 **5万国产卡** 集群

下一阶段问题：如何把 GPU、任务、推理、缓存和多集群资源统一组织起来。

AI Infra 不是一个单点工具，而是一套栈

从基础设施到模型服务

AI Infra 要回答：模型跑在哪里，请求怎么路由，加速卡怎么分配，模型权重怎么分发，质量和成本怎么观测。Kubernetes 提供可扩展的资源和工作负载抽象。



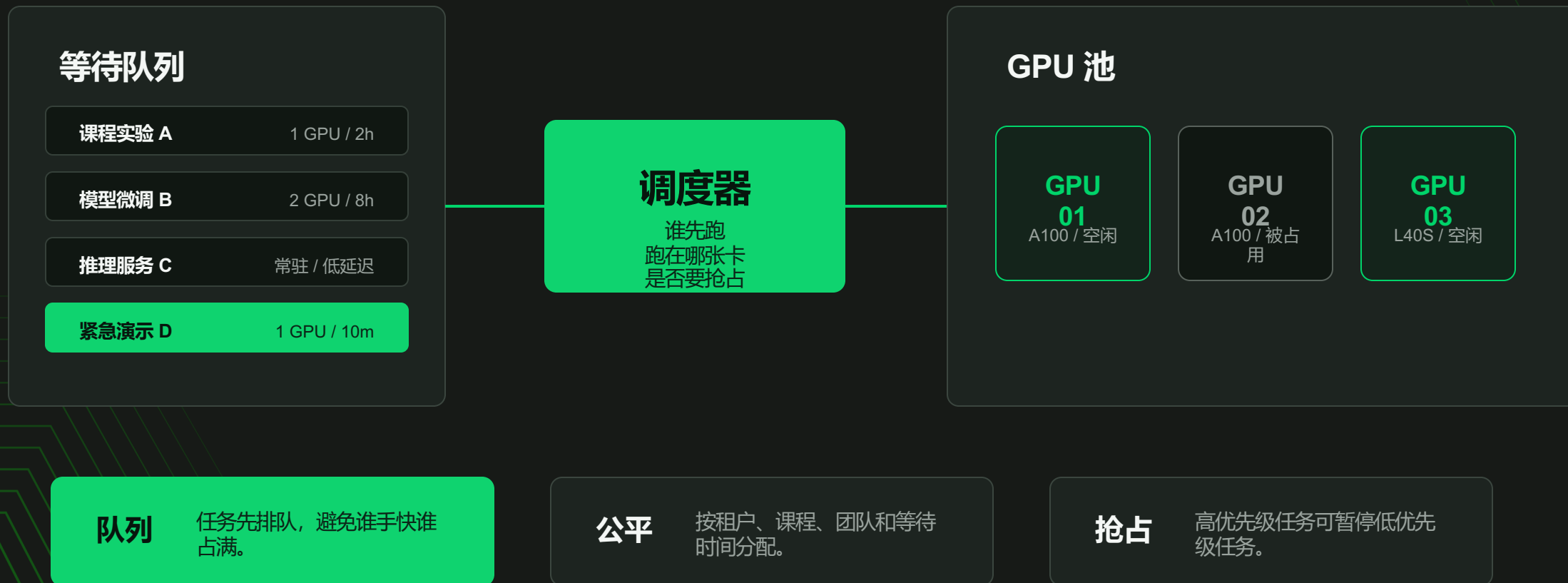


GPU 工作负载让调度问题更难

- 不是所有节点都有 GPU，GPU 型号和拓扑也不同。
- 训练和分布式推理常要求一组 Pod 同时启动。
- 资源不仅是 CPU/内存，还包括显存、NVLink、RDMA 和队列公平性。

GPU 调度就像实验室显卡排队

场景：实验室只有几张显卡，课程实验、论文实验、演示任务都要跑



Kubernetes 如何解决 算力调度与算法优化问题

从多集群资源池、队列准入、Gang、DRA 拓扑，到 vLLM
推理运行时指标闭环

核心判断

AI 平台的竞争力
不是能否拉起 Pod
而是能否让昂贵 GPU
被正确地排队、放置、
隔离和优化

Kubernetes 当前方向：更开放的资源模型和更标准的入口

- DRA 让 GPU、网卡、加速器等设备可以被更精细地描述和分配。
- Kueue、LeaderWorkerSet 等项目在补齐 AI 训练和推理的队列与副本组织能力。
- Gateway API 与 Inference Extension 让模型入口、路由和治理更标准化。
- 可观测、安全、多租户和多集群正在成为 AI 平台的基础能力。

推理服务关注吞吐、延迟和成本

- LLM 推理常见指标：首 token 延迟、每 token 延迟、吞吐量、队列延迟。
- 平台需要在模型质量、成本和响应速度之间做权衡。
- vLLM、SGLang、TensorRT-LLM 等引擎负责高效执行，平台负责调度和治理。



Inference Gateway 让模型调用变成平台能力

- 统一不同模型和推理服务入口。
- 按租户、成本、质量和健康状态做路由。
- 支持限流、鉴权、灰度、观测和失败降级。



RAG 和 Agent 把更多系统接进来

- RAG：检索知识库，把证据放进模型上下文。
- Agent：模型可以调用工具、API、数据库或浏览器完成任务。
- 这意味着权限、审计、沙箱、网络边界和 Trace 都变得更重要。

Agent 能调用工具后，权限边界必须前置

01

Agent 提议

我要读文件、跑命令、调 API

02

权限检查

工具白名单、身份、上下文

03

沙箱执行

隔离文件、网络和进程

04

审计清理

记录调用，释放环境和凭据

文件

只能访问任务目录，不能读用户家目录和密钥。

Shell

高风险命令要禁用、审批或放进隔离环境。

网络 / API

限制目标域名、数据库和云账号权限。

凭据

短期、最小权限，用完自动回收。

关键变化：传统应用的权限通常由开发者写死；Agent 的工具调用更动态，边界要由平台统一托管。

Agent Sandbox 的需求来自不可信执行

01

不可信任务

Agent 可能执行代码、命令和外部工具

02

隔离边界

限制文件、网络、身份和权限

03

可审计运行

记录输入、工具调用、日志和退出原因

04

自动清理

任务结束后释放环境和敏感数据

安全

限制文件、凭据和网络访问。

可复现

每次执行都有可描述的环境。

多租户

不同用户和 Agent 不共享危险状态。

agent-sandbox 和 agent-substrate 关注点不同

agent-sandbox

问题：如何安全地运行 Agent？

方式：用 Kubernetes 的 Pod、身份、存储、网络等能力，提供 Sandbox CRD 和控制器。

适合：每个 Agent 有独立隔离环境。

agent-substrate

问题：如何安全且高效地大规模运行 Agent？

方式：Agent 按需唤醒，空闲挂起，多个 Agent 共享 worker pool。

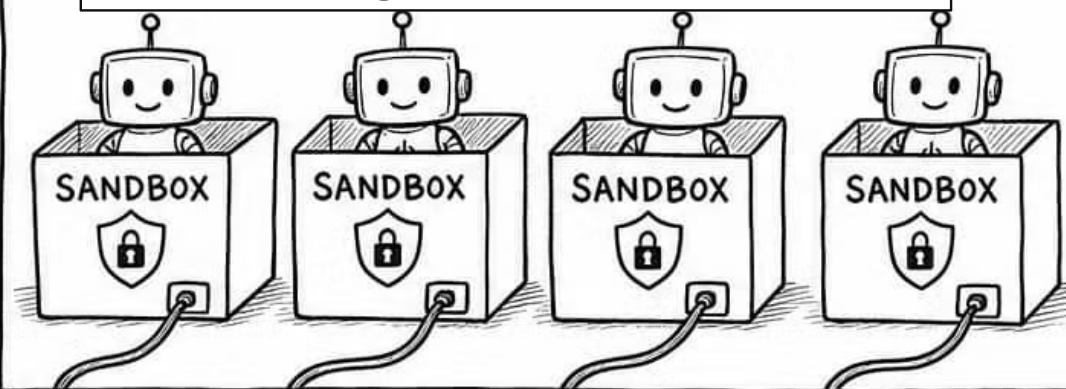
适合：大量低频、短时、可恢复的 Agent。

一句话：sandbox 解决“安全运行”，substrate 进一步解决“安全、低成本、可规模化运行”。

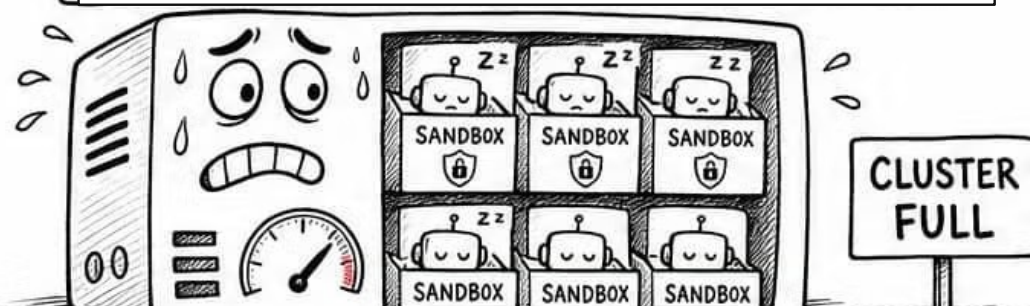
agent-sandbox: 每个 Agent 的安全隔离工作间



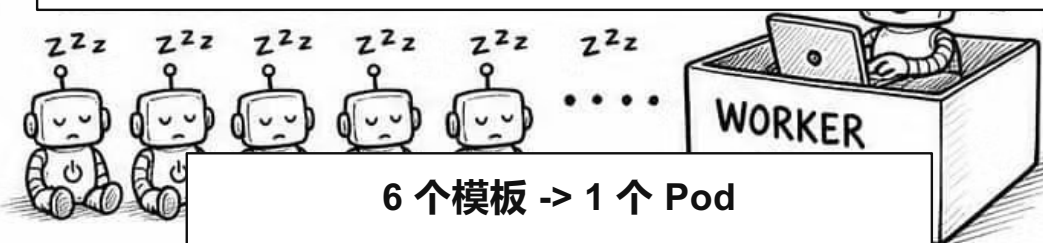
但每个 Agent 都需要自己的 Pod



Agent 多了以后, 资源很快被占满

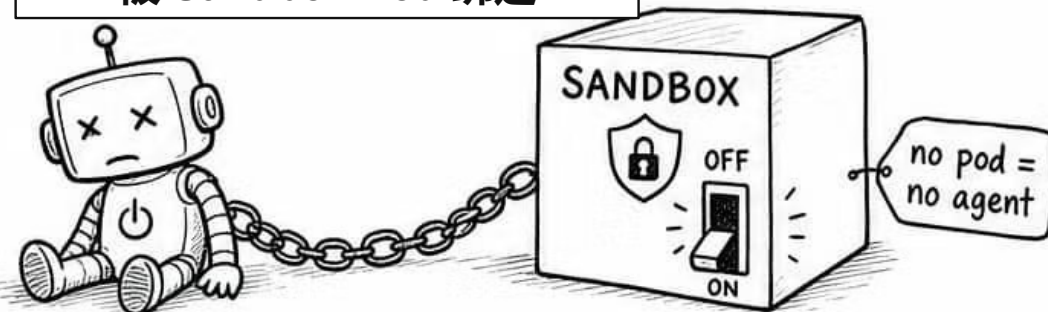


agent-substrate: 按需唤醒, 空闲时挂起



6 个模板 -> 1 个 Pod

Agent 的生命周期
被 Sandbox Pod 绑定



简而言之

agent-sandbox = 安全运行 Agent

agent-substrate = 安全 + 高效规模化



第一部分小结：云原生是现代软件的工程底座

- 容器解决环境一致性和交付问题。
- Kubernetes 解决规模化运行、调度、自愈和扩展问题。
- CNCF 让大量项目在中立社区中协作。
- AI Infra 正把 GPU、推理、Agent Sandbox 和安全治理带入云原生平台。

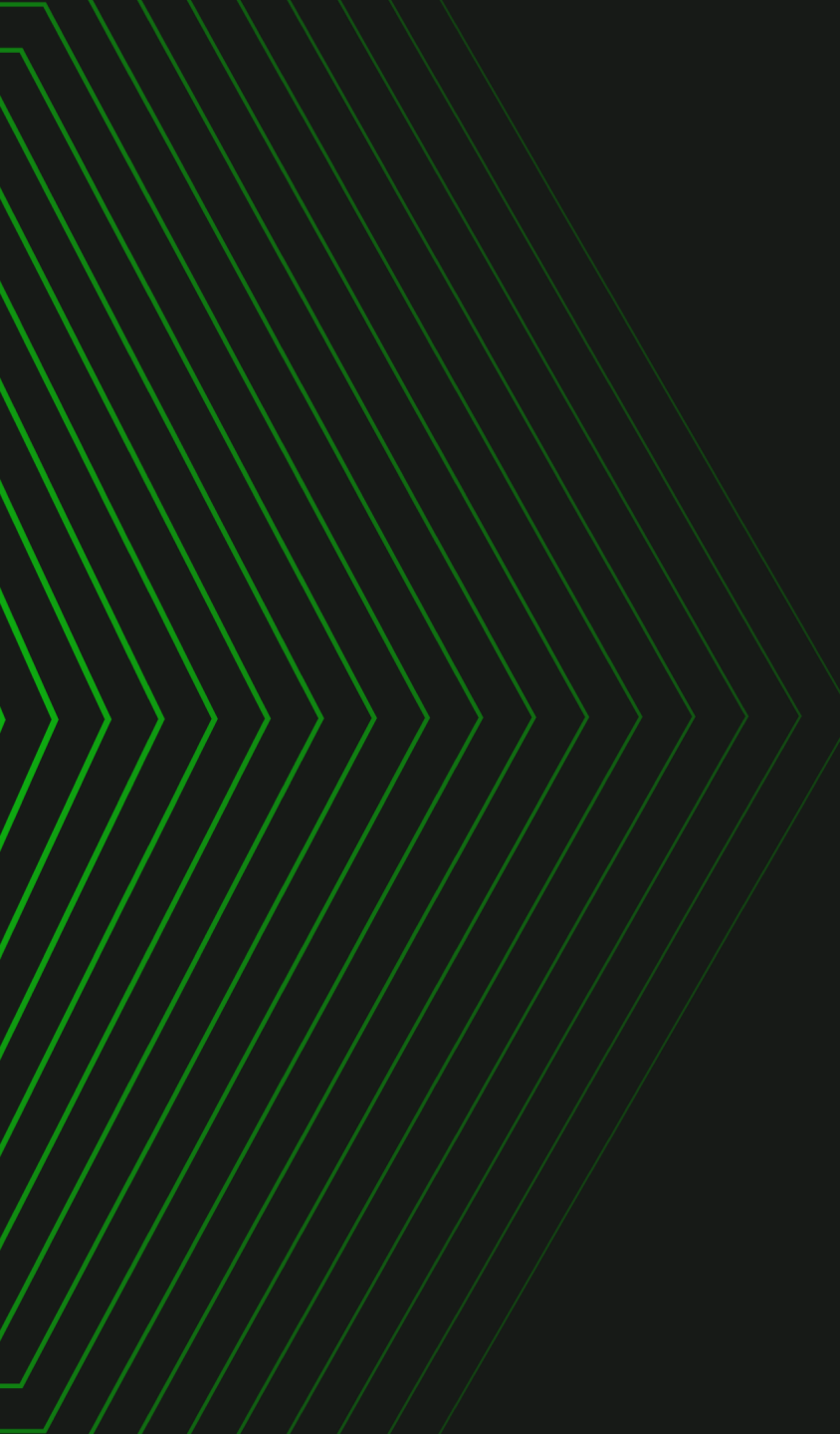
平台在变 开发方式也在变

接下来交给同事介绍 Vibe Coding



从云原生平台到 Vibe Coding

- 云原生降低了运行和交付复杂度。
- AI 工具正在降低编码、调试和原型验证门槛。
- 但真正能长期工作的项目，仍然依赖清晰的协作机制和工程纪律。



Part 02

开源机制与参与

工具会变化，但协作机制会长期重要

互动：你和开源的距离有多近

01

用过开源软件

VS Code、Linux、Python 等都算。

02

有代码托管账号

GitHub、GitLab、Gitee 任意一个。

03

参与过开源项目

Issue、PR、文档、翻译、测试都算。

04

听说过开源之夏

面向学生参与开源社区的项目实践。

05

听过这些名字

Linus、Linux 基金会、Apache 基金会。

如果你举过手，说明你已经在开源生态里。后面讲的是：规则、收益和参与方法。

互动：你怎么判断一个项目是开源

请先按直觉判断：这些说法你是否同意？

开源大模型

开放权重就等于开源吗？

开源 vs 闭源

看到代码就能随使用吗？

看许可

开源 vs 开源社区

有仓库就等于有社区吗？

互动：你怎么判断一个项目是开源

请先按直觉判断：这些说法你是否同意？

开源大模型

开放权重就等于开源吗？

代码、权重、数据、许可证可能开放程度不同。

不一定

开源 vs 闭源

看到代码就能随使用吗？

能否使用、修改、商用、再分发，要看许可证。

看许可

开源 vs 开源社区

有仓库就等于有社区吗？

社区还包括维护者、讨论、治理、发布和安全响应。

不等于

开源不是一个标签，而是一组可验证的规则和持续协作。

下一步看：许可证、社区机制，以及学生如何从使用者变成贡献者。



开源不只是把代码放到网上

- 代码开放只是起点。
- 许可证定义别人能如何使用、修改和分发。
- 公开讨论、公开决策、公开发布，让项目可以被更多人信任和参与。



许可证是开源协作的法律边界

- 宽松许可证：通常更容易被商业产品集成。
- Copyleft 许可证：更强调衍生作品继续开放。
- 企业使用开源时，需要关注许可证、依赖和合规。

许可证选择要看你希望别人怎么使用

不是背许可证名字，而是看你希望别人怎样复用你的项目

MIT

课程项目、小工具、轻量库

尽量简单宽松，方便别人复制、修改、商用。

Apache-2.0

基础设施、企业采用、云原生项目

宽松复用，同时更明确专利授权和贡献边界。

GPL

希望衍生作品继续开放

强调修改和分发后的开放传承，商业集成要更谨慎。

课堂记法：MIT 更简单，Apache-2.0 更适合基础设施协作，GPL 更强调开放延续。

实际项目请阅读 LICENSE 和贡献指南，企业场景还需要合规审查。



开源社区需要透明的治理机制

- 谁能合并代码？谁能发布版本？谁来处理安全漏洞？
- 成熟项目会有维护者、贡献者指南、行为准则和决策流程。
- 规则越清楚，新人越容易参与，企业也越敢依赖。



一个开源项目通常这样协作

- Issue：描述问题、需求或讨论。
- Design：复杂功能先写设计文档。
- Pull Request：提交代码、文档或测试。
- Review：维护者和社区审查。
- Release：打包发布并维护兼容性。

开源能带来什么好处？

—— 从世界杯与足球协作的角度理解开源价值 ——

1



团队协作

像一支球队分工配合，
开发者、测试、文档、用户共同贡献。

2



开放与包容

不分地域和背景，
任何人都可以参与，
好的想法都有机会被看见。

3



公平与透明

规则公开、讨论留痕、
过程可追踪，
就像公开赛制与裁判规则。

4



持续迭代

像赛后复盘和战术升级，
社区让项目一版比一版更强。

5



竞争中合作

不同公司和团队既竞争又共建，
共同推动基础设施进步。

6



成长与领导力

贡献者可以像球员成长为队长一样，
逐步成为 reviewer、maintainer、
leader。



足球因全球协作而精彩，开源因共同创造而强大。

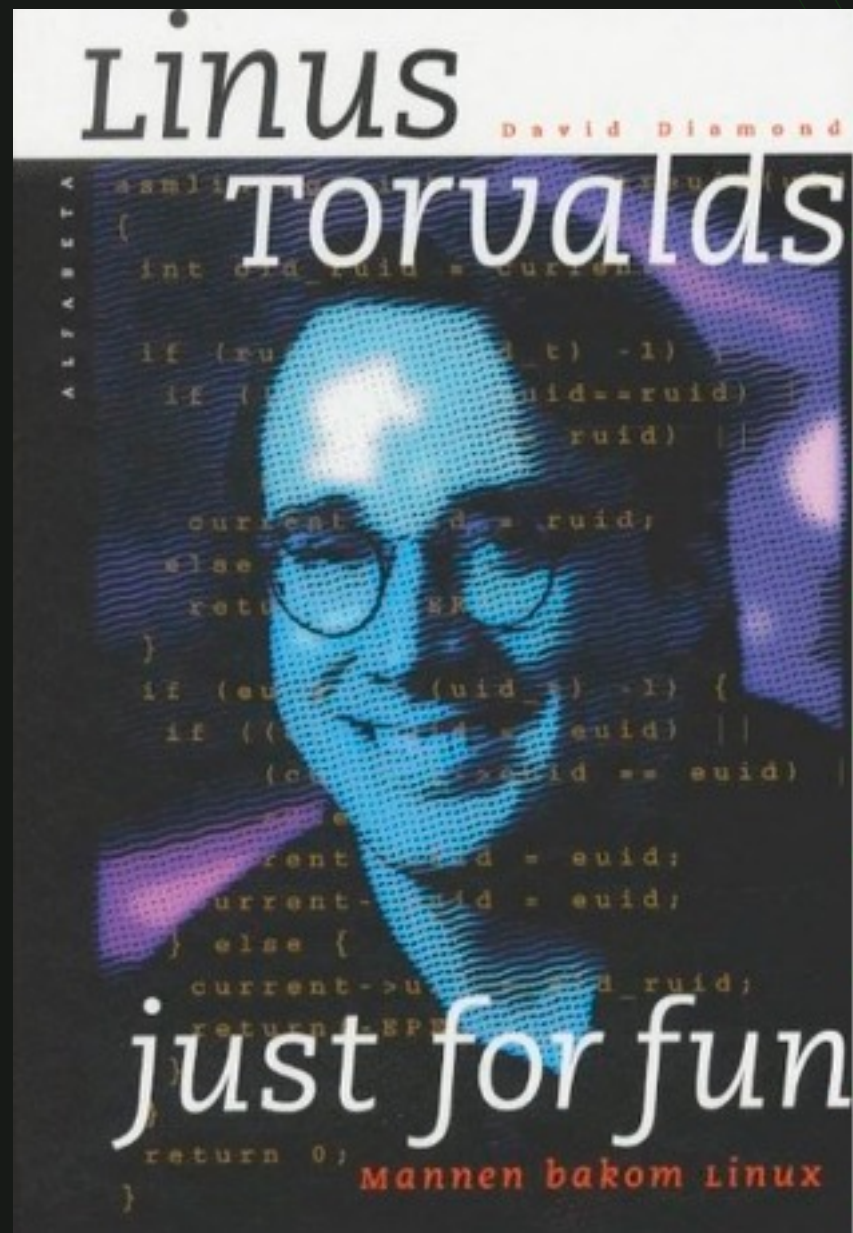


企业为什么愿意投资开源

- 复用社区创新，降低从零开始的成本。
- 把适合通用化的问题放到上游，减少长期维护分叉。
- 建立生态、标准和影响力，吸引用户、伙伴和人才。
- 通过公开协作提升软件质量和安全响应速度。

个人为什么参与开源

- 工作需要
- 职业技能提升与规划
- 学习真实项目的工程实践。
- 和全球开发者交流，获得高质量反馈。
- 积累公开作品和技术信誉。
- 参与自己真正使用或感兴趣的项目。





云原生本身就是开源协作的结果

- Linux、Docker、Kubernetes、Prometheus、Envoy 等项目共同构成基础设施生态。
- CNCF 提供项目生命周期和中立治理框架。
- 云原生的发展速度，很大程度来自跨公司、跨地域的公开协作。

为什么要坚持 Upstream First ?

短期省事，不一定长期省成本

只在下游私改

短期看起来快



版本升级反复改



补丁漂移



维护成本
越来越高



社区难以支持

私有分支
长期维护

下游私改

Upstream First

先上游讨论 / Upstream First

开放协作 共建共赢



充分讨论



Issue/PR
评审



设计对齐

一开始更花时间



确认问题
有通用价值



贡献合并进社区



升级更顺畅



后续更容易
获得支持

文档完善
测试覆盖
持续改进

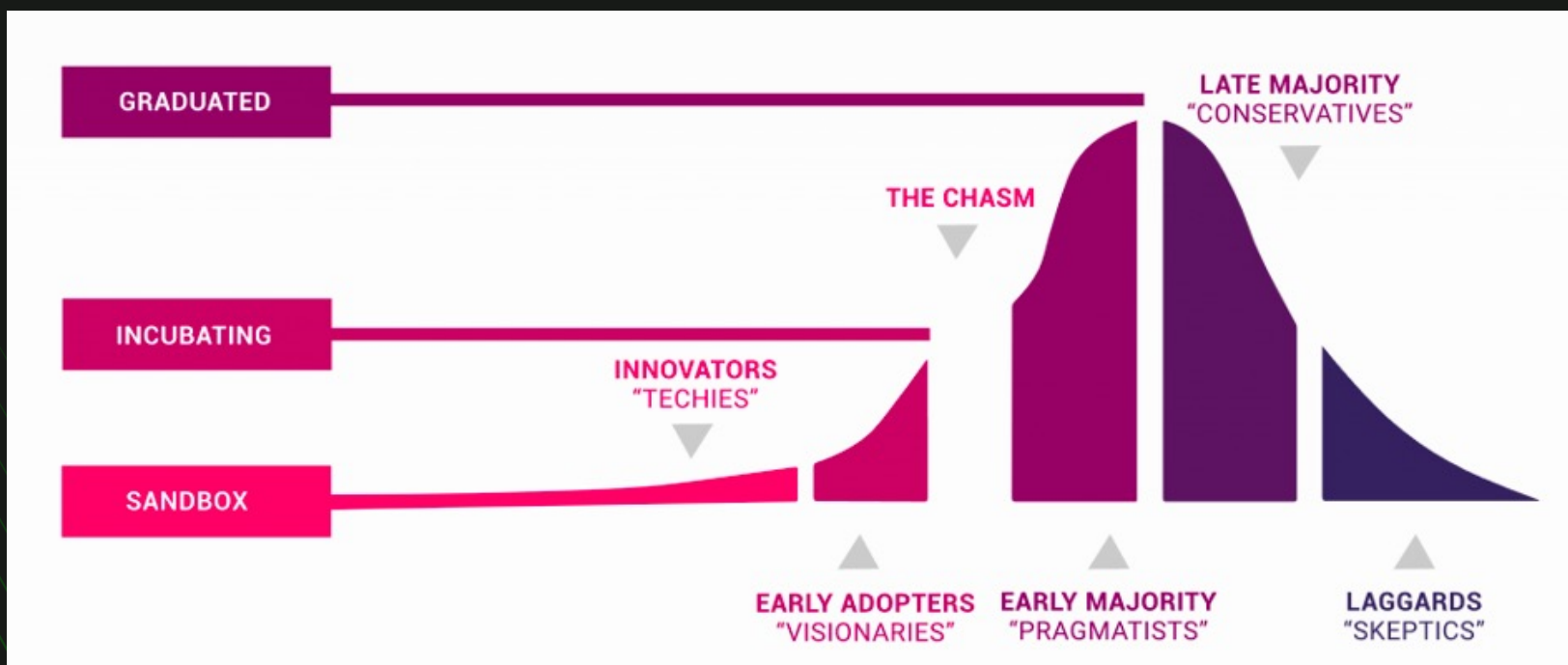


Upstream First = 前期多沟通，后期少还债



看项目健康度，不只看 Star

- 最近是否有稳定提交和发布？
- Issue 和 PR 是否有人响应？
- 贡献者是否多样，还是只靠一两个人？
- 文档、测试、安全流程是否完整？



OpenSSF 让安全最佳实践更可见

- 成熟项目需要清晰的漏洞报告流程。
- 自动化测试、依赖扫描、发布签名和权限管理都很重要。
- 安全不是最后上线前才补的步骤，而是贯穿开发和发布流程。



▼ 基本	13/13 ●
▼ 变更控制	9/9 ●
▼ 报告	8/8 ●
▼ 质量	13/13 ●
▼ 安全	16/16 ●
▼ 分析	8/8 ●

Kubernetes 社区案例：复杂项目如何保持秩序

- SIG 负责不同技术领域，例如 Node、Network、Storage。
- KEP 用来讨论较大功能的动机、设计、兼容性和风险。
- Release Team 负责版本发布节奏和质量门禁。

Community Snapshot - November 2024

94.8k
Contributors

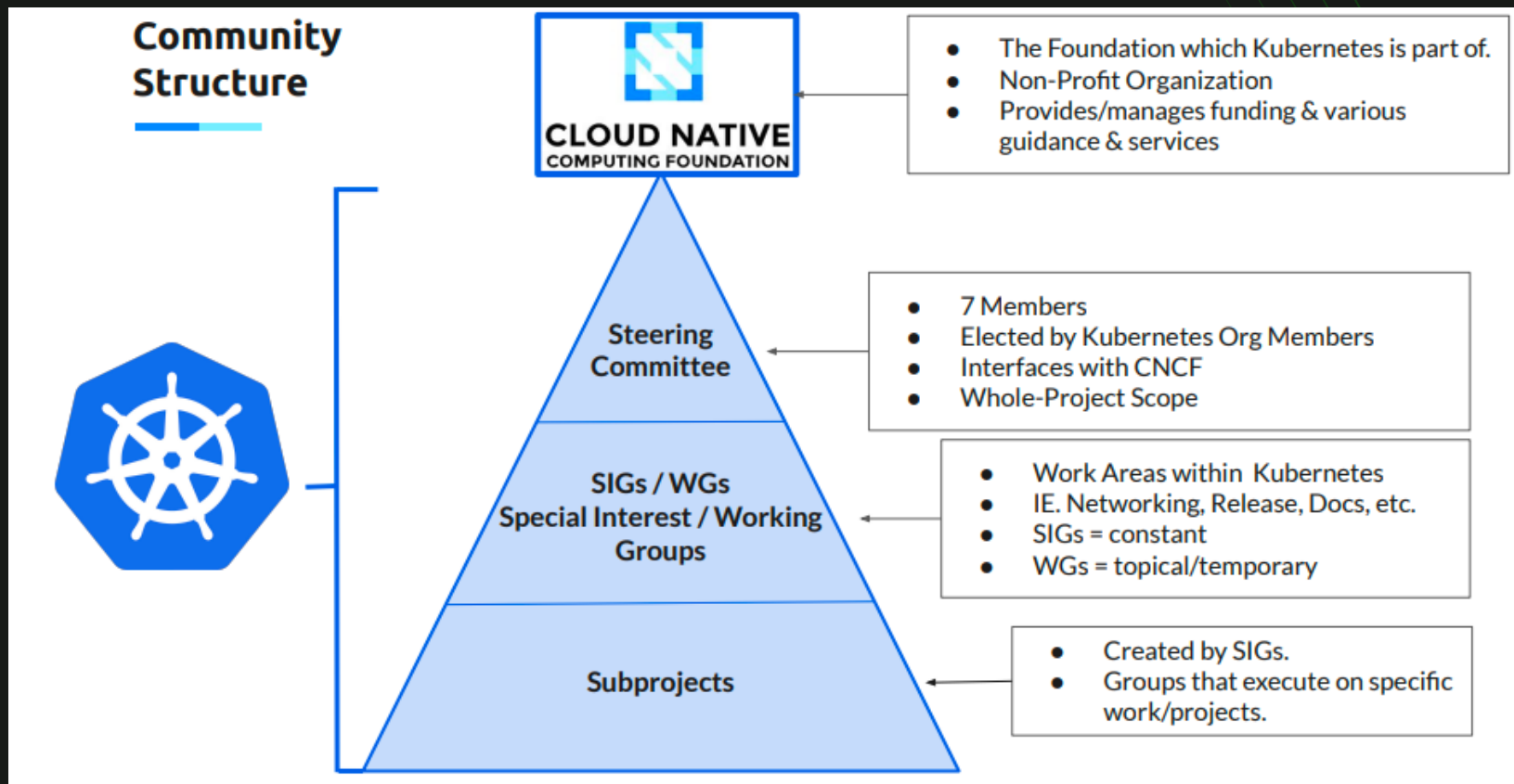
4.56M
Contributions

7.8K
Reviewers

Stats from [Kubernetes Devstats](#), Mar 24 2025

一个 Kubernetes 功能通常这样推进

- 先在 SIG 中提出问题和方向
- 复杂功能写 KEP, 接受设计评审
- 代码 PR 需要测试、Review 和 Approver
- 发布后还要更新文档、博客和用户指南。





参与开源不只有写代码

- 修正文档、翻译、复现实验、补测试。
- 回答用户问题、整理 issue、改进示例。
- 做设计讨论、会议纪要、发布支持、社区活动。
- 对新人来说，非代码贡献常常是最好的入口。

Contributor Ladder: 从旁听到维护者有很多台阶

开源成长不是一跳到核心维护者，而是一层层承担更多责任



从学生视角看，最现实的第一步通常是：参加会议 + 做记录 + 复现问题 + 补文档。



第一次参与可以走这条路径

- 选择你实际使用或愿意长期学习的项目。
- 阅读 README、贡献指南、最近的 Issue 和 PR。
- 先从文档、测试、示例或 good first issue 开始。
- 提问前先搜索，提交前先小范围沟通。



如何提问，才更容易得到回应

- 先搜索已有 Issue，确认不是重复问题。
- 写清环境版本、复现步骤和期望结果。
- 提供最小复现、日志和截图。
- 说明你已经尝试过什么，以及愿意如何继续跟进。



写一个好 PR，也是在练工程能力

- 一个 PR 解决一个清晰问题。
- 说明动机、改动、测试方式和兼容性影响。
- 尊重 Review：解释不同意见，也愿意修改。
- 保持耐心，维护者通常也是志愿或兼职投入。



社区协作的底线：Be kind

- 不同观点可以争论，但不要攻击人。
- 新人犯错很正常，好的社区会帮助人变强。
- 行为准则不是形式，它保护社区长期可持续。



学生可以如何建立开源作品集

- 维护一个公开学习笔记或小工具。
- 给常用项目提交文档、测试或 bugfix。
- 写复盘文章：问题是什么、怎么定位、学到了什么。
- 持续比一次大贡献更重要。



30 天开源实践建议

- 第 1 周：选项目，跑起来，读贡献指南。
- 第 2 周：复现一个 issue，补充日志或文档。
- 第 3 周：提交一个小 PR，认真回应 Review。
- 第 4 周：写一篇复盘，继续关注后续讨论。



推荐学习路径

- 基础工具：Git、GitHub、Markdown、Linux 命令行。
- 云原生入门：Docker、Kubernetes 基本对象、CNCF Landscape 分类。
- 开源参与：贡献指南、Issue/PR、Code Review、Release Notes。
- AI Infra 延伸：推理服务、模型网关、RAG、Agent 工具边界。



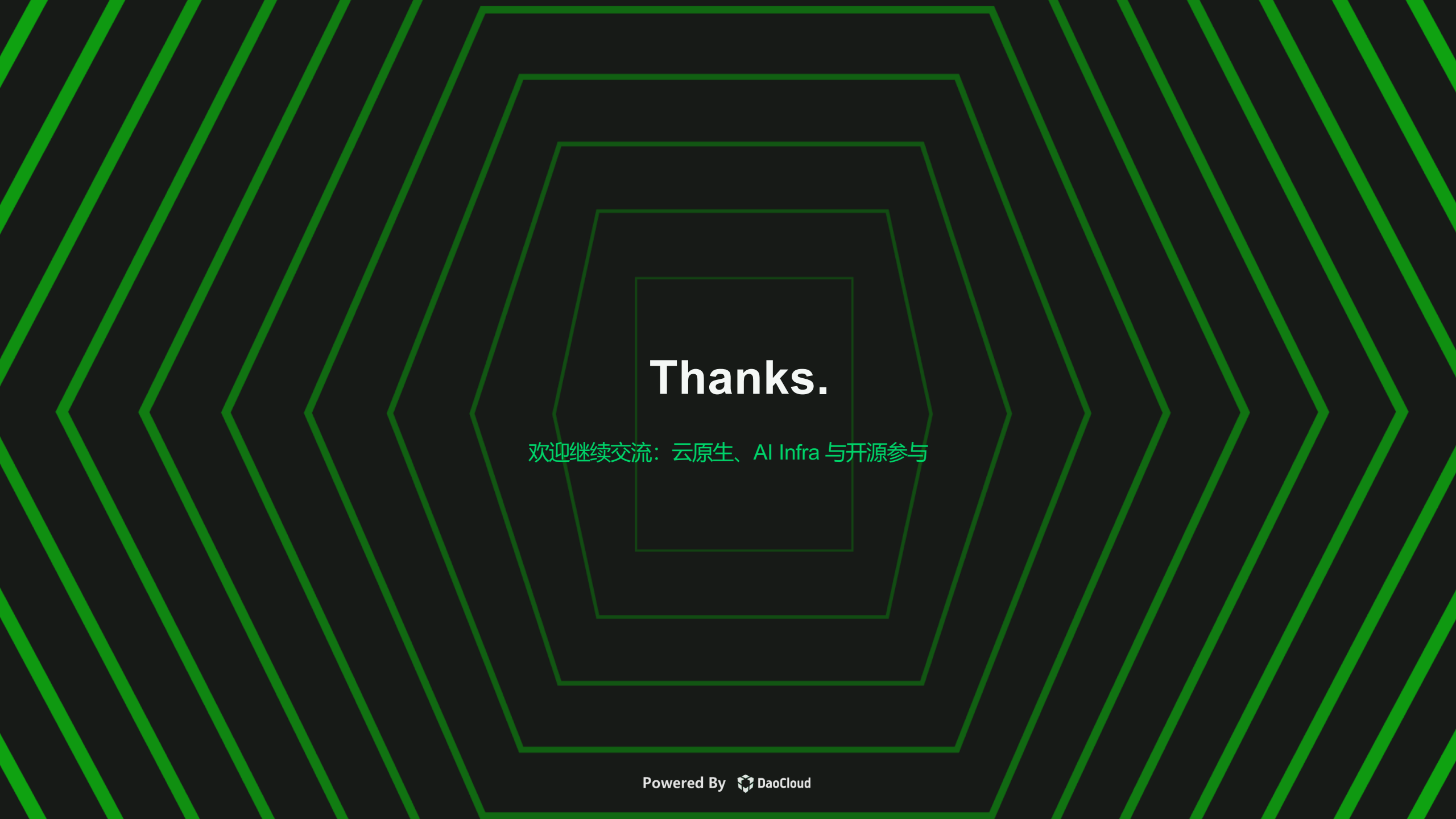
回顾：两条主线其实连在一起

- 云原生回答：现代软件如何被可靠、弹性、可观测地运行。
- AI Infra 回答：AI 工作负载如何进入这套平台。
- 开源回答：这些复杂技术如何跨公司持续演进。
- 对新人来说，理解问题和参与协作比背工具名更重要。



思考题

- 如果一个项目很流行但只有少数维护者，它健康吗？
- 企业应该把哪些改动推到上游，哪些保留在内部？
- AI 写代码越来越容易后，开源社区更需要哪些能力？
- 你这学期能完成的第一个开源贡献是什么？



Thanks.

欢迎继续交流：云原生、AI Infra 与开源参与